Module 6 – Core Java

1. Introduction to Java

- **Question:-1.** History of Java.

Answer:- Java was developed in 1991 by James Gosling at Sun Microsystems under the Green Project.
Initially, it was called Oak, but later renamed Java. It was officially released in 1995 with the feature "Write Once, Run Anywhere (WORA)", allowing programs to run on any platform using JVM.
In 2010, Oracle Corporation acquired Sun Microsystems and now manages Java.

- **Question:-2.** Features of Java (Platform Independent, Object-Oriented, etc.)

Answer:- **1.**Platform Independent – Java programs run on any system using JVM (Write Once, Run Anywhere).
2.Object-Oriented – Based on concepts like class, object, inheritance, polymorphism.
3.Simple – Easy to learn and use compared to other languages.
4.Secure – No direct memory access, provides strong security features.
5.Robust – Handles errors and memory management automatically.
6.Multithreaded – Supports multiple tasks at the same time.
7.High Performance – Faster than many interpreted languages due to JVM.
8.Portable – Can run on different platforms without changes.

- **Question:-3.** Understanding JVM, JRE, and JDK.

Answer:- **1.**JVM (Java Virtual Machine)
Executes Java bytecode and makes Java platform independent.
2.JRE (Java Runtime Environment)
Provides libraries + JVM required to run Java programs.
3.JDK (Java Development Kit)
Full package for development (JRE + compiler + tools).

- **Question:-4.** Setting up the Java environment and IDE (e.g., Eclipse, IntelliJ).

Answer:- Install JDK from official source.

Set PATH environment variable.

Install IDE like Eclipse IDE or IntelliJ IDEA.

Create project → Write code → Compile & Run.

- **Question:-5.** Java Program Structure (Packages, Classes, Methods).

Answer:-

```
package mypackage;   // Package
public class MyClass {   // Class
   public static void main(String[] args) {   // Method
      System.out.println("Hello Java");
   }
}
```

Package – Group of related classes.

Class – Blueprint of objects.

Method – Block of code that performs a task.

2. Data Types, Variables, and Operators

- **Question:-1.** Primitive Data Types in Java (int, float, char, etc.).

Answer:-

1.byte :-1byte

2.short:-2byte

3.int:-4byte

4.long:-8byte

5.float:-4byte

6.double:-8byte

7.char:-1byte

8.boolean:-true/false

- **Question:-2.** Variable Declaration and Initialization.

  Answer:- Declaration :- int a;

  Initialization :- int a=10;

- **Question:-3.** Operators: Arithmetic, Relational, Logical, Assignment, Unary, and Bitwise.

  Answer:-

  Arithmetic: + - * / %

  Relational: == != > < >= <=

  Logical: && || !

  Assignment: = += -= *= /=

  Unary: ++ --

  Bitwise: & | ^ ~ << >>

- **Question:-4.** Type Conversion and Type Casting.

  Answer:- Type Conversion (Automatic):

  Smaller → Bigger

  int a = 10;

  double b = a;

  Type Casting (Manual):

  Bigger → Smaller

  double a = 10.5;

  int b = (int) a;

3. Control Flow Statements

- **Question:-1.** If-Else Statements.

Answer:-

```
int a = 10;
if(a > 5) {
   System.out.println("Greater");
} else {
   System.out.println("Smaller");
}
```

- **Question:-2.** Switch Case Statements.

Answer:-

```
int day = 2;
switch(day) {
   case 1: System.out.println("Monday"); break;
   case 2: System.out.println("Tuesday"); break;
   default: System.out.println("Invalid");
}
```

- **Question:-3.** Loops (For, While, Do-While).

Answer:-

1.For Loop

```
for(int i = 1; i <= 5; i++) {
   System.out.println(i);
}
```

2.While Loop

```
int i = 1;
while(i <= 5) {
   System.out.println(i);
   i++;
}
```

3.Do-While Loop

```
int i = 1;
do {
   System.out.println(i);
   i++;
} while(i <= 5);
```

- **Question:-4.** Break and Continue Keywords.

Answer:-

```
Break:- for(int i=1;i<=5;i++){
   if(i==3) break;
   System.out.println(i);
}
Continue:- for(int i=1;i<=5;i++){
   if(i==3) continue;
   System.out.println(i);
}
```

4. Classes and Objects

- **Question:-1.** Defining a Class and Object in Java.

Answer:-

Class → Blueprint (template) of objects
Object → Instance of a class

```
class Student {
    int id;
    String name;
}
Student s1 = new Student(); // Object creation
```

- **Question:-2.** Constructors and Overloading.

Answer:-

Constructor → Special method used to initialize objects

Same name as class, no return type

```
class Student {
    Student() {
        System.out.println("Default Constructor");
    }
}
```

Constructor Overloading → Multiple constructors with different parameters

```
class Student {
    Student(int id) {}
    Student(int id, String name) {}
```

```
}
```

- **Question:-3.** Object Creation, Accessing Members of the Class.

Answer:-

```
class Student {
    int id;
    void display() {
        System.out.println(id);
    }
}
Student s = new Student(); // create object
s.id = 101;                // access variable
s.display();               // call method
```

- **Question:-4.** this Keyword.

Answer:-

```
class Student {
    int id;

    Student(int id) {
        this.id = id; // current object variable
    }
}
```

5. Methods in Java

- **Question:-1.** Defining Methods.

  Answer:-

  A method is a block of code that performs a specific task.

```
void display() {
    System.out.println("Hello");
}
```

- **Question:-2.** Method Parameters and Return Types.

  Answer:-

  Parameters = input to method , Return type = output

```
int add(int a, int b) {
    return a + b;
}
```

- **Question:-3.** Method Overloading.

  Answer:-

  Same method name, different parameters.

```
int add(int a, int b) { return a+b; }
double add(double a, double b) { return a+b; }
```

- **Question:-4.** Static Methods and Variables.

  Answer:-

  Belong to class (not object)

```
static int x = 10;

static void show() {
    System.out.println(x);
}
```

6. Object-Oriented Programming (OOPs) Concepts

- **Question:-1.** Basics of OOP: Encapsulation, Inheritance, Polymorphism, Abstraction.

  Answer:-

  1. Encapsulation → Data hiding
  2. Inheritance → Reuse code
  3. Polymorphism → Many forms
  4. Abstraction → Hide details

- **Question:-2.** Inheritance: Single, Multilevel, Hierarchical.

  Answer:-

  1.Single Inheritance:- A class inherits properties and methods from one parent class.
  2.Multilevel Inheritance:- A class inherits from a parent, and that parent itself inherits from another class (chain inheritance).
  3.Hierarchical Inheritance:- Multiple classes inherit properties and methods from the same parent class.

- **Question:-3.** Method Overriding and Dynamic Method Dispatch.

  Answer:-

  1. Method Overriding:- When a child class provides its own implementation of a method that is already defined in the parent class.

2. Dynamic Method Dispatch:- It is the process where the method call is decided at runtime, based on the object type (not reference type).

7. Constructors and Destructors

- **Question:-1.** Constructor Types (Default, Parameterized).

  Answer:-

  1. Default Constructor:- A constructor with no parameters. It initializes objects with default values.
  2. Parameterized Constructor:- A constructor that accepts parameters to initialize objects with specific values.

- **Question:-2.** Copy Constructor (Emulated in Java).

  Answer:- A copy constructor is used to create a new object by copying values from another object of the same class.
  In Java, there is no built-in copy constructor, but it can be manually created by passing an object as a parameter to a constructor and copying its data.

- **Question:-3.** Constructor Overloading.

  Answer:- Constructor overloading is when a class has multiple constructors with different parameters (different number or type of arguments).

- **Question:-4.** Object Life Cycle and Garbage Collection.

  Answer:-

  1. Object Life Cycle:- The object life cycle is the process from creation of an object, its use in the program, until it becomes eligible for deletion.
  2. Garbage Collection:- Garbage Collection is the process where Java automatically deletes unused objects from memory to free up space.

8. Arrays and Strings

- **Question:-1.** One-Dimensional and Multidimensional Arrays.

  Answer:- An array that stores elements in a single line (list) using one index.
  An array that stores elements in multiple dimensions (like rows and columns) using more than one index.

- **Question:-2.** String Handling in Java: String Class, StringBuffer, StringBuilder.

  Answer:-

  1. String Class:- A String is an object that represents a sequence of characters and is immutable (cannot be changed after creation).
  2. StringBuffer:- A StringBuffer is a mutable string class, meaning its content can be changed, and it is thread-safe.
  3. StringBuilder:- A StringBuilder is also mutable like StringBuffer, but it is not thread-safe and is faster.

- **Question:-3.** Array of Objects.

  Answer:- An array of objects is an array that stores references to objects instead of primitive data types.

- **Question:-4.** String Methods (length, charAt, substring, etc.).

  Answer:-

  Returns the number of characters in a string.
  Returns the character at a specific index.
  Returns a part of the string from start index to end index.
  Converts the string to uppercase.
  Converts the string to lowercase.
  Compares two strings for exact equality.
  Removes extra spaces from the beginning and end.
  Replaces characters or substrings with new ones.
  Compares two strings ignoring case differences.

9. Inheritance and Polymorphism

- **Question:-1.** Inheritance Types and Benefits.

  Answer:-

  Inheritance allows a class to acquire properties and methods of another class.
  Types:
  Single , Multilevel , Hierarchical
  Benefits:
  Code reusability , Easy maintenance , Improves readability , Supports polymorphism

- **Question:-2.** Method Overriding.

Answer:- Method overriding occurs when a child class provides a specific implementation of a method already defined in the parent class.

- **Question:-3.** Dynamic Binding (Run-Time Polym orphism).

  Answer:- Dynamic binding is when the method call is resolved at runtime, based on the actual object type, not the reference type.

- **Question:-4.** Super Keyword and Method Hiding.

  Answer:-

  Super Keyword:
  The super keyword is used to refer to the parent class, such as calling parent methods or constructors.

  Method Hiding:
  Method hiding occurs when a child class defines a static method with the same name as a static method in the parent class, hiding it instead of overriding.

10. Interfaces and Abstract Classes

- **Question:-1.** Abstract Classes and Methods.

  Answer:- An abstract class is a class that cannot be instantiated and may contain abstract (no body) and concrete methods.
  An abstract method is a method declared without implementation and must be implemented by subclasses.

- **Question:-2.** Interfaces: Multiple Inheritance in Java.

Answer:- An interface is a collection of abstract methods.
Java supports multiple inheritance through interfaces, allowing a class to implement multiple interfaces.

- **Question:-3.** Implementing Multiple Interfaces.

  Answer:- A class can implement more than one interface by using the implements keyword and must provide implementation for all methods.

11. Packages and Access Modifiers

- **Question:-1.** Java Packages: Built-in and User-Defined Packages.

  Answer:- A package is a collection of related classes and interfaces.
  Built-in Packages: Predefined packages provided by Java (e.g., java.lang, java.util).
  User-Defined Packages: Packages created by the user to organize their own classes.

- **Question:-2.** Access Modifiers: Private, Default, Protected, Public.

  Answer:- Access modifiers control the visibility of classes, methods, and variables.
  Private: Accessible only within the same class
  Default: Accessible within the same package
  Protected: Accessible within the same package and subclasses
  Public: Accessible from anywhere

- **Question:-3.** Importing Packages and Classpath.

  Answer:-

  Importing Packages:
  The import keyword is used to access classes from other packages.
  Example: import java.util.Scanner;
  Classpath:
  Classpath is the path where Java looks for class files and packages during compilation and execution.

12. Exception Handling

- **Question:-1.** Types of Exceptions: Checked and Unchecked.

  Answer:-

  Checked Exceptions: Checked at compile-time and must be handled (e.g., IOException).
  Unchecked Exceptions: Occur at runtime and are not checked at compile-time (e.g., ArithmeticException, NullPointerException).

- **Question:-2.** try, catch, finally, throw, throws.

  Answer:-

  try: Block where code that may cause exception is written
  catch: Handles the exception
  finally: Executes always (used for cleanup)
  throw: Used to explicitly throw an exception
  throws: Declares exceptions that a method may throw

- **Question:-3.** Custom Exception Classes.

Answer:-Custom exceptions are user-defined exception classes created by extending the Exception class to handle specific situations.

13. Multithreading

- **Question:-1.** Introduction to Threads.

  Answer:- A thread is a lightweight process that allows a program to run multiple tasks simultaneously (multithreading).

- **Question:-2.** Creating Threads by Extending Thread Class or Implementing Runnable Interface.

  Answer:- Threads can be created in two ways:
  Extending Thread Class: Override the run() method and start the thread
  Implementing Runnable Interface: Implement run() method and pass object to a Thread

- **Question:-3.** Thread Life Cycle.

  Answer:-
  The life cycle of a thread includes:

  New → Runnable → Running → Waiting/Blocked → Terminated

- **Question:-4.** Synchronization and Inter-thread Communication.

  Answer:- Synchronization: Controls access to shared resources to avoid conflicts

Inter-thread Communication: Allows threads to communicate using methods like wait(), notify(), and notifyAll()

14. File Handling

- **Question:-1.** Introduction to File I/O in Java (java.io package).

  Answer:- File I/O in Java is used to read from and write to files using classes from the java.io package.

- **Question:-2.** FileReader and FileWriter Classes.

  Answer:-

  FileReader: Used to read character data from a file
  FileWriter: Used to write character data to a file

- **Question:-3.** BufferedReader and BufferedWriter.

  Answer:-

  BufferedReader: Reads data efficiently using buffering
  BufferedWriter: Writes data efficiently with buffering

- **Question:-4.** Serialization and Deserialization.

  Answer:-

  Serialization: Converting an object into a byte stream to store or send
  Deserialization: Converting byte stream back into an object.

15. Collections Framework

- **Question:-1.** Introduction to Collections Framework.

  Answer:- The Collections Framework is a set of classes and interfaces used to store, manage, and manipulate groups of objects dynamically.

- **Question:-2.** List, Set, Map, and Queue Interfaces.

  Answer:-

  List: Ordered collection, allows duplicates (e.g., ArrayList)
  Set: Unordered collection, no duplicates (e.g., HashSet)
  Map: Stores data in key-value pairs (e.g., HashMap)
  Queue: Follows FIFO (First In First Out) order

- **Question:-3.** ArrayList, LinkedList, HashSet, TreeSet, HashMap, TreeMap.

  Answer:-

  ArrayList: Dynamic array, fast access
  LinkedList: Doubly linked list, efficient insertion/deletion
  HashSet: Stores unique elements, no order
  TreeSet: Stores unique elements in sorted order
  HashMap: Key-value pairs, no order
  TreeMap: Key-value pairs in sorted order

- **Question:-4.** Iterators and ListIterators.

  Answer:-

  Iterator: Used to traverse elements in a collection (forward only)
  ListIterator: Allows forward and backward traversal (only for List)

16. Java Input/Output (I/O)

- **Question:-1.** Streams in Java (InputStream, OutputStream).

  Answer:- Streams are used to handle input and output of data.

  InputStream: Used to read data from a source
  OutputStream: Used to write data to a destination

- **Question:-2.** Reading and Writing Data Using Streams.

  Answer:- Data is read and written using stream methods like read() and write() to transfer data between source and destination.

- **Question:-3.** Handling File I/O Operations.

  Answer:- File I/O operations involve creating, reading, writing, and deleting files using classes like File, FileInputStream, and FileOutputStream.